Normalization Document

Hotel Management System

Document Type	Normalization Document
Project	Hotel Management System
Course	Database Course
Students	Hussein Joudeh, Mohammad Abdulrahman
Date	May 16, 2026
Related Implementation	Java 17, Java Swing, JDBC, MySQL, Maven, FlatLaf

Table of Contents

1. Introduction

This document explains the normalization approach applied to the Hotel Management System database. Normalization is a systematic database design process used to reduce redundancy, avoid update, insertion, and deletion anomalies, and keep related data in the correct tables. The final system uses a relational MySQL schema that supports guest management, room inventory, reservations, packages, services, check-in, check-out, invoices, payments, reports, and user roles.

The design follows First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF). It also includes intentional historical price snapshots for room, package, and service charges so that old reservations and invoices remain accurate even if current prices change later.

2. Overview of Database Tables

The final schema contains 15 base tables. Each table has a focused purpose and participates in a clear set of relationships.

Table	Purpose
Users	Stores staff accounts, roles, active status, and bcrypt password hashes.
Guest	Stores guest contact and personal information.
RoomType	Defines room categories, base prices, capacity, and amenities.
Room	Stores physical rooms, current status, floor, and room type reference.
Reservation	Stores booking header details, guest, dates, status, and confirmation number.
ReservationItem	Stores room assignment, optional package, and price snapshots for a reservation.
Package	Stores optional hotel stay packages.
Service	Stores individual hotel services such as restaurant, laundry, spa, and transport.
PackageService	Junction table that links packages to included services.
ServiceRequest	Stores service requests made for a reservation with service price snapshots.
Invoice	Stores invoice header, total amount, status, and reservation reference.
InvoiceItem	Stores detailed invoice line items and amounts.
Payment	Stores payment transactions for invoices.
GuestDocument	Stores guest identity document details.
RoomStatusLog	Stores audit history of room status changes.

3. First Normal Form (1NF)

A table is in First Normal Form when every column contains atomic values, there are no repeating groups, and each row can be uniquely identified by a primary key. The Hotel Management System satisfies 1NF because each table stores one type of entity or relationship, and multi-valued data is separated into related tables.

- Guest stores one guest per row, with atomic fields such as name, phone, email, nationality, and date of birth.
- Room stores one physical room per row; room type details are not repeated inside the room record.
- ReservationItem separates reserved room/package details from the Reservation header instead of repeating room-related fields in multiple reservation columns.
- PackageService separates package-service inclusions instead of storing multiple service names in one Package field.

Therefore, the database avoids repeating groups and represents each fact in a structured relational form.

4. Second Normal Form (2NF)

Second Normal Form requires the table to be in 1NF and requires all non-key attributes to depend on the whole primary key. This is especially important for tables that represent relationships, such as many-to-many tables or line-item tables.

- PackageService represents the relationship between a package and a service. Its quantity_included attribute depends on the complete package-service relationship, not only on Package or only on Service.
- ReservationItem stores details for a reservation item such as room_id, package_id, room price snapshot, and package price snapshot. These values describe that specific reservation item.
- InvoiceItem stores line-item details such as description, amount, and quantity. These values describe the invoice item itself rather than only the parent invoice.

Because each non-key field is placed in the table where it depends on the key, the design avoids partial dependency problems.

5. Third Normal Form (3NF)

Third Normal Form requires a table to be in 2NF and to avoid transitive dependencies. In other words, non-key attributes should not depend on other non-key attributes. The database satisfies 3NF by separating master data from operational transactions and by using foreign keys to connect related records.

- RoomType stores type name, base price, capacity, and amenities. Room stores only room_type_id, avoiding repeated type data in every room.
- Guest data is stored separately from Reservation, so guest contact details are not duplicated across booking records.
- Invoice is separated from InvoiceItem, so invoice header data and charge details are not mixed in one table.
- Payment is separated from Invoice, which supports multiple payments per invoice and avoids repeated invoice details on each payment row.
- RoomStatusLog stores audit history separately from Room, so the Room table keeps only the current room status.

The schema intentionally stores historical snapshots such as ReservationItem.price, ReservationItem.package_price_snapshot, and ServiceRequest.price. These snapshot columns preserve historical billing values and do not duplicate mutable master data for normal update purposes.

6. Table-by-Table Normalization Notes

Table	Normalization Note
Users	Stores one staff account per row. Role and active status belong directly to the user. Passwords are stored as hashes.
Guest	Stores one guest per row with atomic personal/contact fields. Reservations and documents are separated.
RoomType	Centralizes room category data to prevent repeated prices and capacity values on each Room.
Room	References RoomType and stores current room-specific information only.
Reservation	Stores booking header data. Room/package details are stored in ReservationItem.
ReservationItem	Stores reservation item details and historical price snapshots.
Package	Stores package master data only; included services are in PackageService.
Service	Stores individual service master data. Requests and package inclusions are separated.
PackageService	Resolves the Package-Service many-to-many relationship and stores included quantity.
ServiceRequest	Stores a service request for a reservation with quantity, status, notes, and price snapshot.
Invoice	Stores invoice header data and total status; line details are in InvoiceItem.
InvoiceItem	Stores detailed charge rows for an invoice.
Payment	Stores one payment transaction per row, linked to Invoice.
GuestDocument	Stores identity document records linked to Guest. Document numbers are kept unique.
RoomStatusLog	Stores status change audit records linked to Room and Users.

7. Many-to-Many Relationship Normalization

The relationship between Package and Service is many-to-many: one package may include several services, and one service may be included in several packages. A direct design with multiple service columns in Package would break 1NF and create update problems. The normalized solution is the PackageService junction table.

Relationship	Normalized Solution
Package M:N Service	PackageService(package_id, service_id, quantity_included)
Benefit	Prevents repeating service columns, supports many inclusions, and allows included quantity to belong to the relationship.

8. Historical Price Snapshot Justification

Hotel prices can change over time. If an old invoice depended only on the current RoomType, Package, or Service price, historical totals would become incorrect after a price update. For this reason, the database stores price snapshots at the time of the operation.

Snapshot Field	Reason
ReservationItem.price	Stores the room price used when the reservation item was created.
ReservationItem.package_price_snapshot	Stores the package price used for the reservation.
ServiceRequest.price	Stores the service unit price at the time of the request.

These snapshots support accurate check-out invoices and reports while still keeping the master tables normalized for current operational data.

9. Avoided Anomalies

- Update anomaly avoided: Changing a RoomType price or capacity is done once in RoomType, not repeated across every Room.
- Insertion anomaly avoided: A new Service can be added before it is assigned to any Package or requested by any guest.
- Deletion anomaly reduced: PackageService links can be deleted without deleting the related Package or Service master record.
- Historical billing protected: Old invoices remain accurate because room, package, and service prices are stored as snapshots.
- Payment consistency maintained: Payments are stored separately from Invoice, and the payment trigger updates invoice status based on total paid.

10. Conclusion

The Hotel Management System database is organized around normalized relational tables that satisfy 1NF, 2NF, and 3NF. The design separates guests, rooms, reservations, services, packages, invoices, payments, documents, logs, and users into focused tables connected by foreign keys. Junction tables resolve many-to-many relationships, while snapshot fields protect historical financial records. As a result, the database reduces redundancy, avoids common anomalies, and provides a reliable foundation for the Java Swing JDBC application.

11. References

- database/schema.sql - final tables, keys, constraints, and indexes.
- database/seed.sql - demonstration data used for testing and presentation.
- database/procedures.sql - stored procedures used by the system.
- database/triggers.sql - payment trigger for invoice status updates.
- database/views.sql - reporting and dashboard views.
- Project_Report.pdf - final project report.
- Database course materials and standard normalization principles for 1NF, 2NF, and 3NF.